Scenario-Based SQL Questions – Set 5

Practical SQL scenarios • Questions and logic explanations

Scenario 1: Identify Top Performing Employees

Question:
Find the top 3 employees who made the most sales in the last 6 months.

Logic Explanation:

Join the employees table with the sales table using employee_id (the employee who made the sale).

Filter sales of the last 6 months using WHERE sale_date >= CURDATE() - INTERVAL 6 MONTH.

Use SUM(amount) with GROUP BY employee to calculate the total sales of each employee.

Use ORDER BY total_sales DESC to list the highest performers first.

Use LIMIT 3 to restrict the output to the top 3 employees.

Concepts Used:

Joins (INNER JOIN)

Aggregation (SUM())

Sorting (ORDER BY)

Limiting Results (LIMIT)

Scenario 2: Detect Revenue Decline

Question:
Find products whose sales dropped by more than 40% compared to the previous quarter.

Logic Explanation:

Calculate the total sales of each product for each quarter using SUM(amount) with GROUP BY product_id, YEAR(sale_date) and QUARTER(sale_date).

Perform a self-join on the quarterly result using product_id.

Match each quarter with its previous quarter (same year and quarter - 1, or previous year's Q4 when the quarter is Q1).

Calculate the percentage drop: (previous_quarter - current_quarter) / previous_quarter * 100.

Filter products where the drop is greater than 40%.

Concepts Used:

Self Join

Aggregation (SUM())

Date Functions (YEAR(), QUARTER())

Filtering (WHERE)

Scenario 3: Track Consistent Customers

Question:
Identify customers who bought something every month for the last 12 months.

Logic Explanation:

Filter sales of the last 12 months using WHERE sale_date >= CURDATE() - INTERVAL 12 MONTH.

Use DATE_FORMAT(sale_date, '%Y-%m') to get the purchase month.

Apply GROUP BY customer_id and count the distinct purchase months.

Use HAVING COUNT(DISTINCT DATE_FORMAT(sale_date, '%Y-%m')) = 12 to keep customers who purchased in every month.

Join with the customers table to display the customer name.

Concepts Used:

Date Functions (DATE_FORMAT(), INTERVAL)

Aggregation (COUNT(DISTINCT))

Grouping (GROUP BY)

Filtering Groups (HAVING)

Scenario 4: Monitor Inventory Management

Question:
Find products with low stock that need to be reordered.

Logic Explanation:

Select product_name and stock_quantity from the products table.

Use a WHERE clause to filter products whose stock_quantity is below the reorder level (for example, less than 5 units).

Use ORDER BY stock_quantity ASC so that the products with the lowest stock appear first.

Concepts Used:

Filtering (WHERE)

Sorting (ORDER BY)

Scenario 5: Predict Potential Fraudulent Transactions

Question:
Identify customers who made unusually high purchases compared to their regular spending in the last 3 months.

Logic Explanation:

Use a subquery to calculate each customer's average purchase amount (regular spending) with AVG(amount) and GROUP BY customer_id.

Join the sales table with this subquery using customer_id.

Filter transactions of the last 3 months using WHERE sale_date >= CURDATE() - INTERVAL 3 MONTH.

Flag transactions where the amount is much higher than the customer's average, for example amount > 2 * average amount.

Display the customer name, transaction amount and their average spending.

Concepts Used:

Subqueries

Aggregate Functions (AVG())

Joins (INNER JOIN)

Date Functions (CURDATE(), INTERVAL)